

浙江大学

本科实验报告

课程名称： 电子工程训练（甲）

姓 名： 李彦萱

学 院： 信息与工程学院

系：

专 业： 电子科学与技术

学 号： 3250100761

指导教师： 施红军 叶险峰 邓靖靖

2026 年 4 月 17 日

浙江大学实验报告

专业：电子科学与技术

姓名：李彦萱

学号：3250100761

日期：2026.6.14

地点：浙江大学

课程名称：电子工程训练（甲） 指导老师：施红军 叶险峰 邓靖靖 成绩：_____

实验名称：智能小车实验 实验类型：__ 同组学生姓名：__ 林放__

一、实验目的

1. 了解智能小车的机械结构及主要元器件的功能与原理，包括 L293D 电机驱动芯片、H 桥驱动电路、红外传感器模块、WiFi 通信模块等。
2. 掌握 Arduino 平台下通过 PWM 信号控制直流电机转速与转向的方法，理解 analogWrite() 函数与占空比的关系。
3. 学习红外传感器的基本原理，掌握利用红外传感器实现自动循迹和避障的算法逻辑。
4. 理解 WiFi 串口通信协议，掌握上位机与下位机之间的指令编码、传输与解码流程，实现小车远程遥控。
5. 培养模块化编程思维，能够将基础动作（前进、后退、左转、右转、刹车）封装为函数，组合构建复杂的花式动作序列。。

二、实验任务与要求

2.1 小车组装

完成底盘机械组装，正确安装电机、轮子、电池盒及驱动扩展板。

2.2 前进后退以及花式动作

编写基本前进、后退、刹车、左转、右转函数，使用 PWM 信号调节速度，验证小车基本运动功能。在基本动作函数基础上，通过循环与延时调用组合，实现花式动作序列。

2.3 红外循迹实验

通过红外循迹传感器返回 HIGH 和 LOW 的不同值实现小车循迹。

2.4 红外避障实验

通过红外避障传感器，使小车在前方无障碍时直行，右侧有障碍时左转避让，左侧有障碍时右转避让，两侧均有障碍时后退并旋转掉头。

2.5 WiFi 遥控实验

连接 WiFi 模块，实现串口通信协议的数据接收与指令解码。使用上位机发送指令，实现前进、后退、左转、右转、停止、调速以及模式切换的远程控制。

三、实验方案设计与实验参数计算（3.1 实验方案总体设计、3.2 各功能电路设计与计算、3.3 完整的实验电路……）

四、主要仪器设备

使用 Arduino UNO、L293D 扩展板、直流电机、红外循迹传感器模块、红外避障传感器模块、WiFi 模块、智能小车底盘套件、电池组等仪器设备。

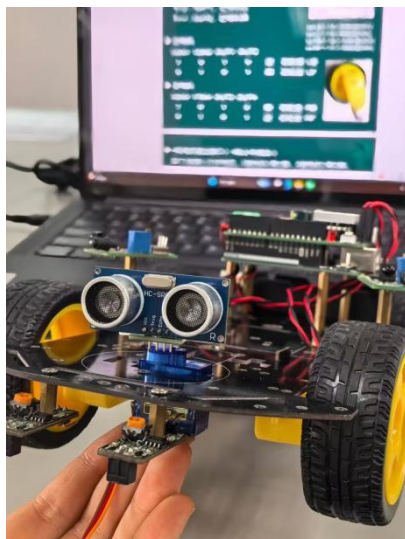
实验名称：智能小车实验 姓名：李彦萱 学号：3250100761

五、实验步骤、实验调试过程、实验数据记录

5.1 实验步骤

5.1.1 小车载装

根据步骤先后组装电机、轮子、Arduino UNO 开发板、L293D 扩展板、电池盒、红外循迹传感器模块、红外避障传感器模块、超声波模块。



5.1.2 前进后退以及花式动作

电机方向由 `digitalWrite` 控制，转速由 `analogWrite` 输出的 PWM 占空比控制。每个动作函数接受 `time` 参数，实际执行时间为 $\text{time} \times 100 \text{ ms}$ 。而花式动作通过 `for` 循环控制重复次数，配合不同时间参数产生多样化的运动模式。

部分代码如下：

(1) 基本动作

```
void run(int time) // 前进
{
    digitalWrite(Right_motor_go, HIGH); // 右电机前进
    digitalWrite(Right_motor_back, LOW);
    analogWrite(ENA, 150);
    digitalWrite(Left_motor_go, HIGH); // 左电机前进
    digitalWrite(Left_motor_back, LOW);
    analogWrite(ENB, 150);
    delay(time * 100); // 执行时间
}

void back(int time) // 后退
{
    digitalWrite(Right_motor_go, LOW); // 右轮后退
    digitalWrite(Right_motor_back, HIGH);
    analogWrite(Right_motor_go, 0);
    analogWrite(Right_motor_back, 150);
    digitalWrite(Left_motor_go, LOW); // 左轮后退
```

```
digitalWrite(Left_motor_back, HIGH);
analogWrite(Left_motor_go, 0);
analogWrite(Left_motor_back, 150);
delay(time * 100);
}
void brake(int time) // 刹车，停车
{
digitalWrite(Right_motor_go, LOW);
digitalWrite(Right_motor_back, LOW);
digitalWrite(Left_motor_go, LOW);
digitalWrite(Left_motor_back, LOW);
delay(time * 100);
}
void left(int time) // 左转
{
digitalWrite(Right_motor_go, HIGH);
digitalWrite(Right_motor_back, LOW);
analogWrite(Right_motor_go, 150);
analogWrite(Right_motor_back, 0);
digitalWrite(Left_motor_go, LOW);
digitalWrite(Left_motor_back, LOW);
analogWrite(Left_motor_go, 0);
analogWrite(Left_motor_back, 0);
delay(time * 100);
}
void right(int time) // 右转
{
digitalWrite(Right_motor_go, LOW);
digitalWrite(Right_motor_back, LOW);
analogWrite(Right_motor_go, 0);
analogWrite(Right_motor_back, 0);
digitalWrite(Left_motor_go, HIGH);
digitalWrite(Left_motor_back, LOW);
analogWrite(Left_motor_go, 150);
analogWrite(Left_motor_back, 0);
delay(time * 100);
}
void spin_left(int time) // 原地左旋
{
digitalWrite(Right_motor_go, HIGH);
digitalWrite(Right_motor_back, LOW);
analogWrite(Right_motor_go, 200);
analogWrite(Right_motor_back, 0);
```

```
digitalWrite(Left_motor_go, LOW);
digitalWrite(Left_motor_back, HIGH);
analogWrite(Left_motor_go, 0);
analogWrite(Left_motor_back, 150);
delay(time * 100);
}
void spin_right(int time) // 原地右旋
{
digitalWrite(Right_motor_go, LOW);
digitalWrite(Right_motor_back, HIGH);
analogWrite(Right_motor_go, 0);
analogWrite(Right_motor_back, 150);
digitalWrite(Left_motor_go, HIGH);
digitalWrite(Left_motor_back, LOW);
analogWrite(Left_motor_go, 150);
analogWrite(Left_motor_back, 0);
delay(time * 100);
}
```

(2) 小车花式动作

```
void loop()
{
int i;
delay(2000); // 延时 2s 后启动
run(10);
back(10); // 全速前进急停后退
brake(5);
for(i = 0; i < 5; i++)
{
run(10); // 小车间断性前进 5 步
brake(1);
}
for(i = 0; i < 5; i++)
{
back(10); // 小车间断性后退 5 步
brake(1);
}
for(i = 0; i < 5; i++)
{
left(10); // 大弯套小弯连续左旋转
spin_left(5);
```

```

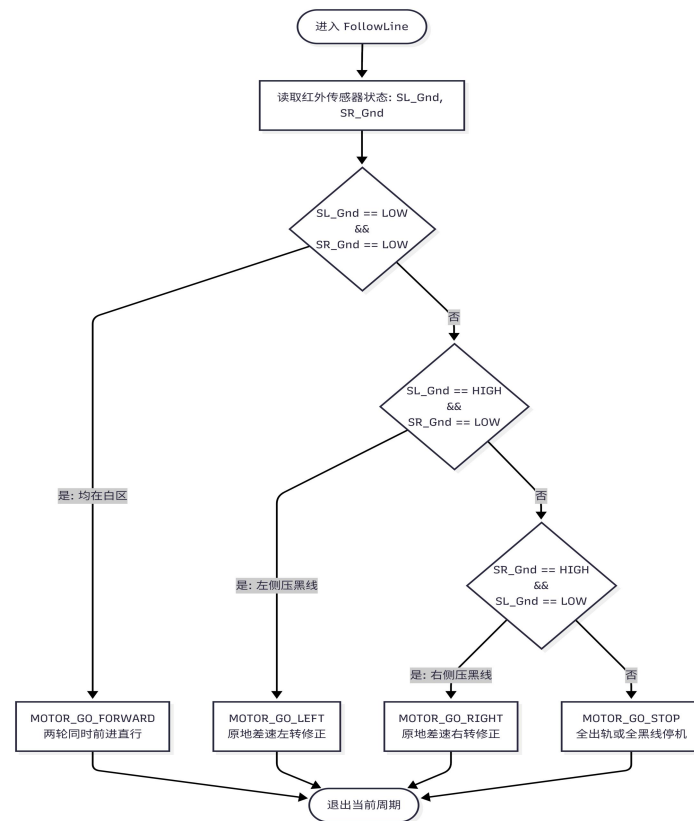
}
for(i = 0; i < 5; i++)
{
    right(10); // 大弯套小弯连续右旋转
    spin_right(5);
}
for(i = 0; i < 10; i++)
{
    right(1); // 间断性原地右转弯
    brake(1);
}
for(i = 0; i < 10; i++)
{
    left(1); // 间断性原地左转弯
    brake(1);
}
for(i = 0; i < 10; i++)
{
    left(3); // 走 S 形前进
    right(3);
}
for(i = 0; i < 10; i++)
{
    spin_left(3); // 间断性原地左打转
    brake(3);
}
for(i = 0; i < 10; i++)
{
    spin_right(3); // 间断性原地右打转
    brake(3);
}
}
    
```

5.1.3 红外循迹

小车运动逻辑如下：

左侧传感器	右侧传感器	判定	动作
LOW	LOW	两侧均在黑线上	直行前进
HIGH	LOW	左侧偏离白区	左转纠偏
LOW	HIGH	右侧偏离白区	右转纠偏
HIGH	HIGH	完全脱离轨迹	停车

红外传感器在黑线上方时接收不到反射光，输出低电平(LOW)；在白色地面时接收到反射光，输出高电平(HIGH)。据此判断小车相对黑色轨迹的位置。



循迹循环代码如下：

```

void loop()
{
  keysacn(); // 调用按键扫描函数，按下按键后启动
  while(1)
  {
    // 有信号为 LOW 没有信号为 HIGH
    SR = digitalRead(SensorRight);
    // 有信号表明在白色区域，车子底板上 L3 亮；
    // 没信号表明压在黑线上，车子底板上 L3 灭
    SL = digitalRead(SensorLeft);
    // 有信号表明在白色区域，车子底板上 L2 亮；
    // 没信号表明压在黑线上，车子底板上 L2 灭
    if (SL == LOW && SR == LOW)
      run(); // 两侧都在黑线上 → 直行
    else if (SL == HIGH && SR == LOW)
      left(); // 左侧偏离黑线：左转纠偏
    else if (SR == HIGH && SL == LOW)
      right(); // 右侧偏离黑线：右转纠偏
    else
      brake(); // 均脱离黑线：停车
  }
}
  
```

5.1.4 红外避障

小车运动逻辑如下：

左避障(SL_2)	右避障(SR_2)	判定	动作
HIGH	HIGH	前方无障碍	直行前进
HIGH	LOW	右侧有障碍	左转避让
LOW	HIGH	左侧有障碍	右转避让
LOW	LOW	前方有障碍	后退+旋转掉头

避障主循环代码如下：

```
void loop()
{
  keysacn(); // 调用按键扫描函数
  while(1)
  {
    // 有信号为 LOW 没有信号为 HIGH
    SR_2 = digitalRead(SensorRight_2);
    SL_2 = digitalRead(SensorLeft_2);
    if (SL_2 == HIGH && SR_2 == HIGH)
      run(); // 前方无障碍 → 直行
    else if (SL_2 == HIGH && SR_2 == LOW)
      left(); // 右侧有障碍 → 左转避让
    else if (SR_2 == HIGH && SL_2 == LOW)
      right(); // 左侧有障碍 → 右转避让
    else // 两侧均有障碍物
    {
      back(4.5); // 后退
      spin_right(4.5); // 旋转掉头
    }
  }
}
```

红外避障传感器发射红外光，当障碍物在检测距离内时，反射光被接收，输出低电平(LOW)；无障碍时输出高电平(HIGH)。

5.1.5 WiFi 遥控小车

此实验中将小车速度设定为 11 档，不用像以往实验一样修改程序进行调速。

具体如下：

```
int Left_Speed[11] = {90, 106, 122, 138, 154, 170, 186, 203, 218, 234, 255};
int Right_Speed[11] = {90, 106, 122, 138, 154, 170, 186, 203, 218, 234, 255};
```


实验名称：智能小车实验 姓名：李彦萱 学号：3250100761

在通讯上使用指令包格式为 FF + [3 字节数据] + FF，各字节含义如下：

字节	含义	示例
buffer[0]	指令类型	0x00=电机，0x02=调速，0x13=模式
buffer[1]	子命令	0x01=前进，0x02=后退，0x03=左转，0x04=右转
buffer[2]	参数值	调速时表示档位(0~10)，模式时表示目标模式

通讯部分代码如下：

(1) 串口指令接收与帧解析：

```
void Get_uartdata(void)
{
    static int i;
    if (Serial.available() > 0) // 判断串口缓冲器是否有数据装入
    {
        serial_data = Serial.read(); // 读取串口
        if(rec_flag == 0)
        {
            if(serial_data == 0xff) // 检测帧头
            {
                rec_flag = 1;
                i = 0;
                Costime = 0;
            }
        }
        else
        {
            if(serial_data == 0xff) // 检测帧尾
            {
                rec_flag = 0;
                if(i == 3)
                {
                    Communication_Decode(); // 收到完整3字节，解码
                }
                i = 0;
            }
            else
            {
                buffer[i] = serial_data;
                i++;
            }
        }
    }
}
```

```
}
```

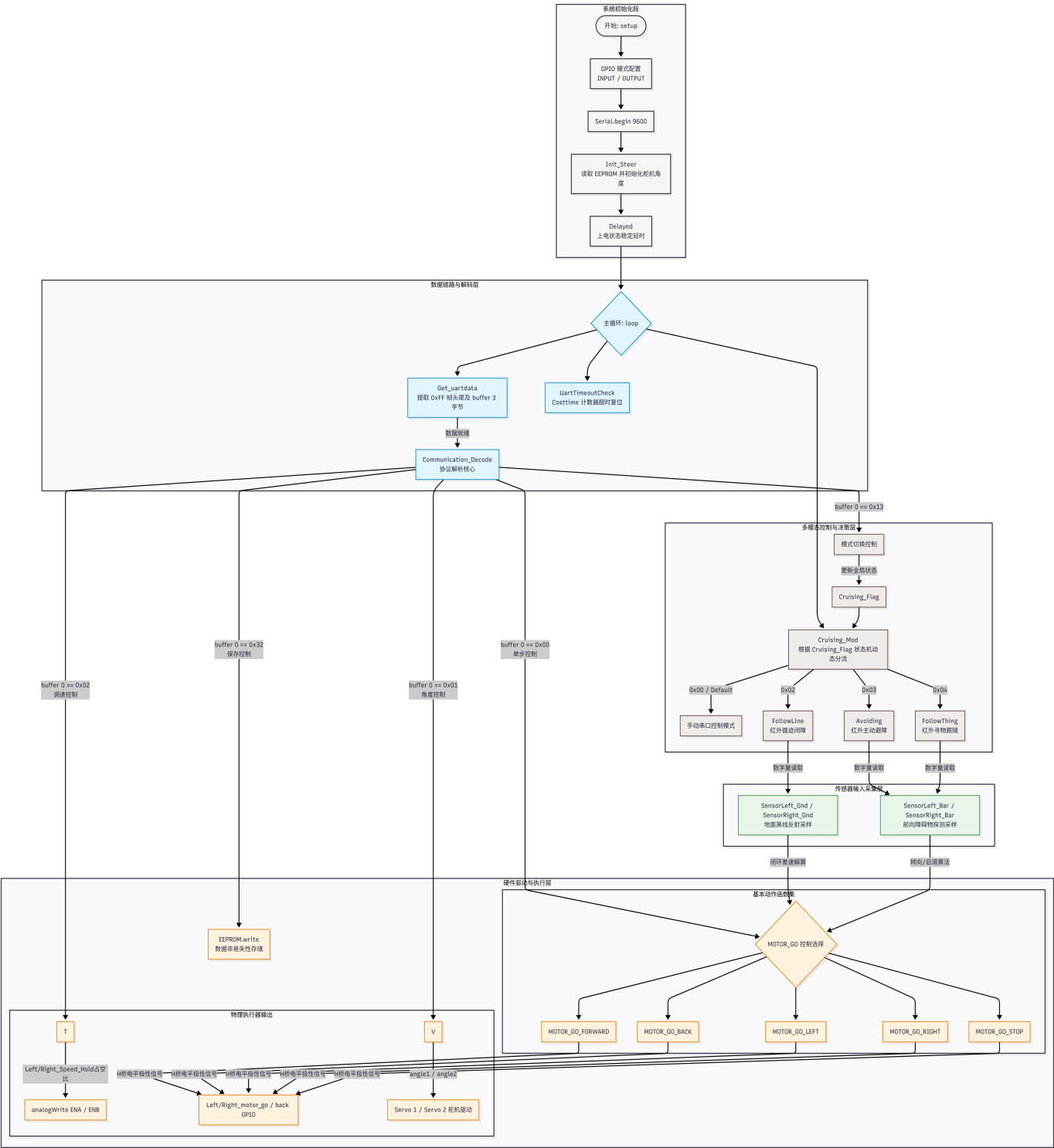
(2) 指令解码：

```
void Communication_Decode()
{
    if(buffer[0] == 0x00) // 电机控制命令
    {
        switch(buffer[1])
        {
            case 0x01: MOTOR_GO_FORWARD; return;
            case 0x02: MOTOR_GO_BACK; return;
            case 0x03: MOTOR_GO_LEFT; return;
            case 0x04: MOTOR_GO_RIGHT; return;
            case 0x00: MOTOR_GO_STOP; return;
            default: return;
        }
    }
    else if(buffer[0] == 0x02) // 调速命令
    {
        int i, j;
        if(buffer[2] > 10) return;
        if(buffer[1] == 0x01) // 左侧调档
        {
            i = buffer[2];
            Left_Speed_Hold = Left_Speed[i];
            analogWrite(ENB, Left_Speed_Hold);
        }
        if(buffer[1] == 0x02) // 右侧调档
        {
            j = buffer[2];
            Right_Speed_Hold = Right_Speed[j];
            analogWrite(ENA, Right_Speed_Hold);
        }
    }
    else if(buffer[0] == 0x13) // 模式切换命令
    {
        switch(buffer[1])
        {
            case 0x02: Cruising_Flag = 2; return; // 巡线模式
            case 0x03: Cruising_Flag = 3; return; // 避障模式
            case 0x04: Cruising_Flag = 4; return; // 超声波避障模式
```

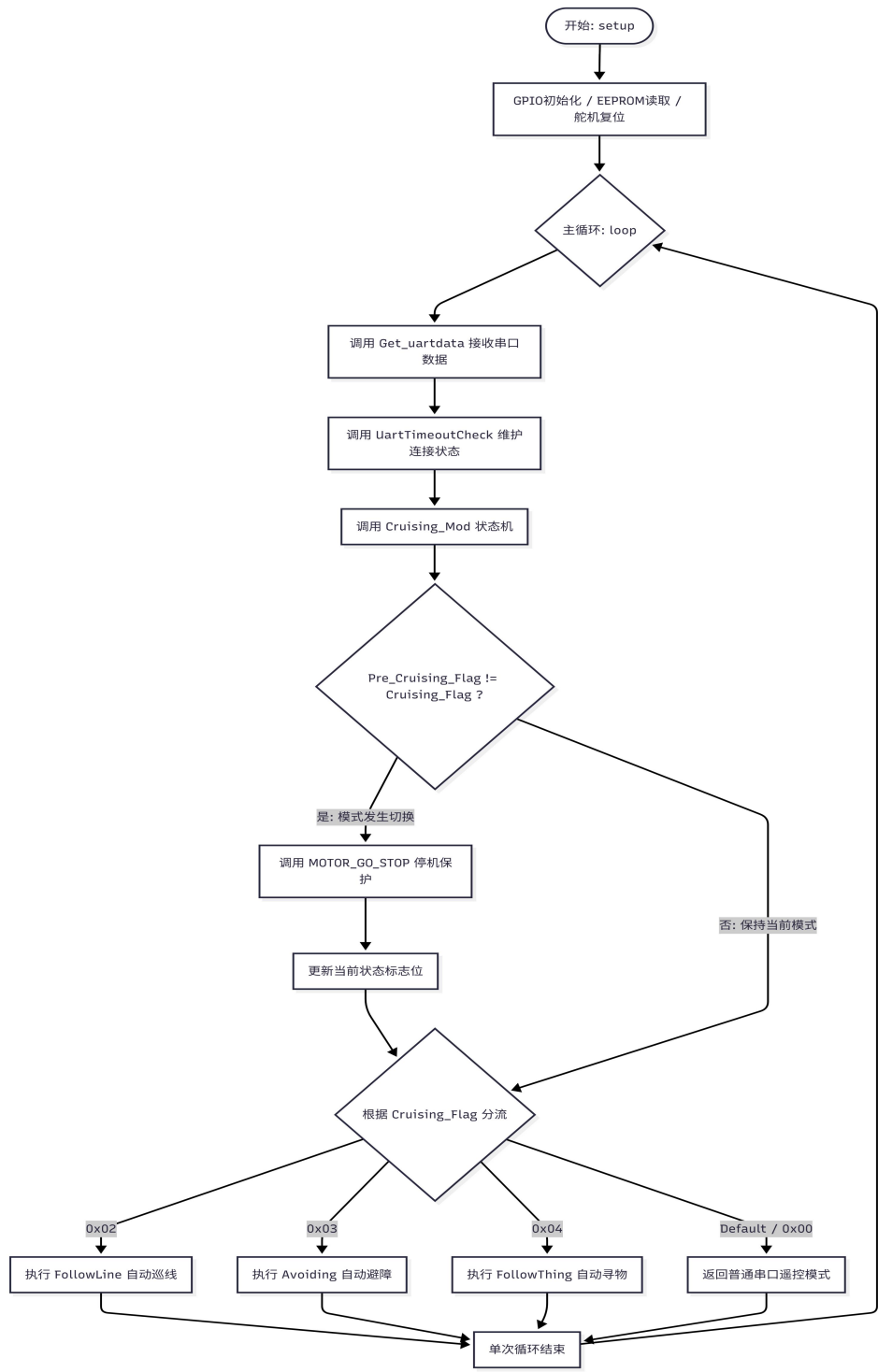
```
case 0x00: Cruising_Flag = 0; return; // 正常遥控模式
default:  Cruising_Flag = 0; return;
}
}
}
```

代码结构图如下（使用 Mermaid 编辑器，图像生成代码放在最后）：

（1）系统总体架构：



(2) 主循环与状态机流程图



5.2 实验调试过程

在进行智能小车的各阶段实验中主要遇到问题如下:

5.2.1 循迹时小车在弯道处容易冲出轨迹:

调低速度, 最终在 110 左右可以正常过弯。

实验名称：智能小车实验 姓名：李彦萱 学号：3250100761

5.2.2 红外传感器检测不稳定，出现撞墙问题：
通过不断调整红外传感器上的电位器反复实验，最终实现功能。

六、实验结果和分析处理

6.1 基础运动与花式动作

成功实现前进、后退、刹车、左转、右转、原地左旋、原地右旋等基本动作以及花式动作序列。且通过修改 `analogWrite` 的参数可明显改变车速。

6.2 红外循迹

小车能够在黑色胶带轨迹上自主循线行驶，遇到弯道时自动调整方向。

6.3 红外避障

实现了前方无障碍时小车持续直行。单侧遇到障碍物时能向无障碍侧转向避让，两侧均检测到障碍物时自动后退原地右旋掉头，避障后继续前行。

6.4 WiFi 遥控

上位机连接 WiFi 模块热点后，可远程发送前进/后退/左转/右转/停止指令，小车响应正常。左右轮独立调速功能正常，模式切换功能正常，可在遥控中切换至巡线模式、避障模式。并且舵机记忆和运动正常。

七、讨论、心得

7.1 关于 H 桥电机驱动：

通过本次实验，我深入理解了 H 桥电路控制电机正反转的原理。以左电机为例，`INPUT1` 和 `INPUT2` 的四种电平组合（00=刹车、01=正转、10=反转、11=急停）即可切换电机转向。`L293D` 芯片内部集成两组 H 桥，配合 `Arduino` 数字 IO 与 PWM 输出，实现了两个直流电机的独立方向与速度控制。

7.2 关于 PWM 调速：

`analogWrite` 输出的 PWM 信号通过改变占空比调节施加在电机上的等效电压，从而控制转速。但是数值过低会无法正常运动

7.3 关于传感器信号处理：

红外传感器返回开关量（HIGH/LOW），逻辑简洁但高度依赖环境。循迹实验中传感器离地距离、环境光强度、黑线材质与颜色深度都会影响检测准确性。这让我认识到实际应用中传感器标定与阈值选择的必要性。

7.4 关于模块化编程的好处：

在程序中将 `run()`、`back()`、`left()`、`right()` 等基本动作封装为独立函数后，花式动作只需组合调用并传入不同时间参数即可，代码清晰易调用。WiFi 遥控代码将串口接收、指令解码、模式执行分层处理。

7.6 关于硬件调试的体会：

每个硬件之间存在巨大的差异性，在遇到故障时需要长期耐心的调试，而微小的变化就会

实验名称：智能小车实验 姓名：李彦萱 学号：3250100761

产生较大的影响，要合理控制，比如进行电位器调节，不可以让小车在红外避障时过近或者过远就开始转弯。同时参数的设置也至关重要，比如在我的小车实际运行程序中发现左前轮电机摩擦较大，可以在程序中适当调大 ENA。

附录：

Mermaid 编辑器代码

(1) 巡线决策树：

```
graph TD
    NodeStart([进入 FollowLine]) --> Sampling[读取红外传感器状态: SL_Gnd, SR_Gnd]
    Sampling --> Judge1{SL_Gnd == LOW <br> && <br> SR_Gnd == LOW}
    Judge1 -- 是: 均在白区 --> Action1[MOTOR_GO_FORWARD <br> 两轮同时前进直行]
    Judge1 -- 否 --> Judge2{SL_Gnd == HIGH <br> && <br> SR_Gnd == LOW}
    Judge2 -- 是: 左侧压黑线 --> Action2[MOTOR_GO_LEFT <br> 原地差速左转修正]
    Judge2 -- 否 --> Judge3{SR_Gnd == HIGH <br> && <br> SL_Gnd == LOW}
    Judge3 -- 是: 右侧压黑线 --> Action3[MOTOR_GO_RIGHT <br> 原地差速右转修正]
    Judge3 -- 否 --> Action4[MOTOR_GO_STOP <br> 全出轨或全黑线停机]
    Action1 --> NodeEnd([退出当前周期])
    Action2 --> NodeEnd
    Action3 --> NodeEnd
    Action4 --> NodeEnd
```

(2) 系统总体架构：

```
graph TD
    %% 样式定义
    classDef initStyle fill:#f5f5f5,stroke:#333,stroke-width:2px;
    classDef commStyle fill:#e1f5fe,stroke:#0288d1,stroke-width:1.5px;
    classDef modeStyle fill:#efebee,stroke:#5d4037,stroke-width:1.5px;
    classDef logicStyle fill:#e8f5e9,stroke:#388e3c,stroke-width:1.5px;
    classDef hardwareStyle fill:#fff3e0,stroke:#f57c00,stroke-width:1.5px;

    %% 1. 初始化层
    subgraph Initialization_Layer [系统初始化段]
        A([开始: setup]) --> B[GPIO 模式配置<br>INPUT / OUTPUT]
        B --> C[Serial.begin 9600]
        C --> D[Init_Steer<br>读取 EEPROM 并初始化舵机角度]
        D --> E[Delayed<br>上电状态稳定延时]
    end

    %% 2. 串口通信与数据链路层
    subgraph Communication_Layer [数据链路及解码层]
        E --> F{主循环: loop}
        F --> G[Get_uartdata<br>提取 0xFF 帧头尾及 buffer 3 字节]
    end
```

```
F --> H[UartTimeoutCheck<br>Costime 计数器超时复位]

G -->|数据就绪| I[Communication_Decompile<br>协议解析核心]
end

%% 3. 核心分流决策层
subgraph Decision_Layer [多模态控制与决策层]
    I -->|buffer 0 == 0x13| J[模式切换控制]
    J -->|更新全局状态| K[Cruising_Flag]

    F --> L[Cruising_Mod<br>根据 Cruising_Flag 状态机动态分流]
    K --> L

    %% 模式分流
    L -->|0x00 / Default| M[手动串口控制模式]
    L -->|0x02| N[FollowLine<br>红外循迹闭障]
    L -->|0x03| O[Avoiding<br>红外主动避障]
    L -->|0x04| P[FollowThing<br>红外寻物跟随]
end

%% 4. 数据采样与传感器层
subgraph Sensor_Layer [传感器输入采集层]
    N -->|数字量读取| Q[SensorLeft_Gnd / SensorRight_Gnd<br>地面黑线反射采样]
    O -->|数字量读取| R[SensorLeft_Bar / SensorRight_Bar<br>前向障碍物探测采样]
    P -->|数字量读取| R
end

%% 5. 驱动与硬件执行层
subgraph Actuator_Layer [硬件驱动与执行层]
    %% 串口直接控制与自动模式算法共同驱动动作函数
    I -->|buffer 0 == 0x00<br>单步控制| S
    I -->|buffer 0 == 0x01<br>角度控制| V
    I -->|buffer 0 == 0x02<br>调速控制| T
    I -->|buffer 0 == 0x32<br>保存控制| U[EEPROM.write<br>数据非易失性存储]

    Q -->|闭环差速解算| S
    R -->|转向/后退算法| S

    subgraph Motion_Functions [基本动作函数集]
        S{MOTOR_GO 控制选择}
        S --> MOTOR_GO_FORWARD
        S --> MOTOR_GO_BACK
        S --> MOTOR_GO_LEFT
    end
end
```

```

        S --> MOTOR_GO_RIGHT
        S --> MOTOR_GO_STOP
    end

    subgraph Hardware_Output [物理执行器输出]
        T -->|Left/Right_Speed_Hold 占空比| PWM[analogWrite ENA / ENB]
        MOTOR_GO_FORWARD & MOTOR_GO_BACK & MOTOR_GO_LEFT &
        MOTOR_GO_RIGHT & MOTOR_GO_STOP -->|H 桥电平极性信号| DRIVER[Left/Right_motor_go /
        back GPIO]
        V -->|angle1 / angle2| SERVO[Servo 1 / Servo 2 舵机驱动]
    end
end

%% 样式绑定
class A,B,C,D,E initStyle;
class F,G,H,I commStyle;
class J,K,L,M,N,O,P modeStyle;
class Q,R logicStyle;

class
S,T,U,V,MOTOR_GO_FORWARD,MOTOR_GO_BACK,MOTOR_GO_LEFT,MOTOR_GO_RIGHT,
MOTOR_GO_STOP,PWM,DRIVER,SERVO hardwareStyle;

```

(3) 主循环与状态机流程图

```

graph TD
    Start([开始: setup]) --> Init[GPIO 初始化 / EEPROM 读取 / 舵机复位]
    Init --> Loop{主循环: loop}

    Loop --> CallUart[调用 Get_uartdata 接收串口数据]
    CallUart --> CallTimeout[调用 UartTimeoutCheck 维护连接状态]
    CallTimeout --> CallCruise[调用 Cruising_Mod 状态机]

    CallCruise --> CheckFlag{Pre_Cruising_Flag != Cruising_Flag ?}
    CheckFlag -- 是: 模式发生切换 --> StopRobot[调用 MOTOR_GO_STOP 停机保护]
    StopRobot --> UpdateFlag[更新当前状态标志位]
    UpdateFlag --> SwitchMode{根据 Cruising_Flag 分流}

    CheckFlag -- 否: 保持当前模式 --> SwitchMode

    SwitchMode -->|0x02| Mode2[执行 FollowLine 自动巡线]
    SwitchMode -->|0x03| Mode3[执行 Avoiding 自动避障]
    SwitchMode -->|0x04| Mode4[执行 FollowThing 自动寻物]
    SwitchMode -->|Default / 0x00| ModeDefault[返回普通串口遥控模式]

```


实 验 名 称：智能小车实验 姓 名：李彦萱 学 号：3250100761

Mode2 --> EndLoop[单次循环结束]

Mode3 --> EndLoop

Mode4 --> EndLoop

ModeDefault --> EndLoop

EndLoop --> Loop

装
订
线